Happy Balance Web App Roadmap

Goal

Build a **mobile-first web app** inspired by the uploaded storyboard, using:

- **React + Vite + TypeScript**
- **Tailwind CSS**
- **shadcn/ui**
- **Google Sheets** as the database
- **Google Apps Script** as the lightweight backend/API
- **Cloudflare Pages** for free frontend hosting
- **VS Code + Codex** for step-by-step implementation

This roadmap is written so Codex can follow it **one step at a time**.

Build Strategy

We will build in this order:

1. Create the frontend project
2. Set up Tailwind and base UI
3. Create app layout and routing
4. Build pages with mock data first
5. Create Google Sheets schema
6. Create Google Apps Script API
7. Connect frontend to the API
8. Deploy frontend to Cloudflare Pages
9. Polish mobile UX
10. Add next features later

Project Scope for MVP

Focus only on the first usable MVP.

MVP Pages

- Home / Dashboard
- Goals
- Daily Log
- Profile

MVP Features

- View summary cards
- View goals list

- Add/edit goals
- Add daily log
- View simple profile data
- Read/write data from Google Sheets via Apps Script

Features to defer

- Real authentication
- Complex permissions
- Online counseling
- Notifications
- Rich analytics
- Full storyboard coverage

Recommended Folder Structure

```
happy-balance-web/
  src/
    app/
      layout/
      routes/
    components/
      ui/
      shared/
    features/
      home/
      goals/
      daily-log/
      profile/
    hooks/
    lib/
      utils.ts
      constants.ts
    services/
      api.ts
      goals.service.ts
      logs.service.ts
      profile.service.ts
    types/
      models.ts
      api.ts
    main.tsx
    App.tsx
```

Data Model

Use these Google Sheets tabs.

Sheet: `users`

Columns: - id - email - full_name - phone - created_at - updated_at

Sheet: `goals`

Columns: - id - user_id - title - category - target_value - current_value - status - created_at - updated_at

Sheet: `daily_logs`

Columns: - id - user_id - log_date - mood - energy - stress - note - created_at - updated_at

For MVP, hardcode a demo user id: - `demo-user-001`

UI Direction

Target feel: - calm - soft - modern - mobile-first - app-like

Suggested visual style

- Rounded cards
- Clean spacing
- Soft shadow
- Large tap targets
- Fixed bottom nav on mobile
- Warm pastel palette

Core UI blocks

- Header with greeting
- Summary cards
- Goal cards with progress bar
- Daily log form
- Profile card
- Bottom navigation

Step-by-Step Execution Plan

Step 1 — Create the project

Goal

Initialize the React app.

Manual tasks

Run:

```
npm create vite@latest happy-balance-web -- --template react-ts
cd happy-balance-web
npm install
```

Codex prompt

```
Create a clean React + Vite + TypeScript project structure for a mobile-first
wellness web app called Happy Balance. Keep the starter minimal and ready for
routing, Tailwind, and feature-based folders.
```

Done when

- Project runs locally with `npm run dev`
- TypeScript React app is working

Step 2 — Install Tailwind and base dependencies

Goal

Prepare the UI foundation.

Manual tasks

Install dependencies:

```
npm install tailwindcss @tailwindcss/vite
npm install lucide-react clsx tailwind-merge class-variance-authority
```

Codex prompt

```
Set up Tailwind CSS in a React + Vite + TypeScript project. Add a clean global stylesheet and prepare a mobile-first design foundation.
```

Done when

- Tailwind classes work
- Base typography and spacing are visible

Step 3 — Add routing and app shell

Goal

Create the app skeleton.

Manual tasks

Install router:

```
npm install react-router-dom
```

What to build

- App layout
- Page container
- Bottom navigation
- Routes for Home, Goals, Daily Log, Profile

Codex prompt

```
Create a mobile-first app shell using React Router. Add routes for Home, Goals, Daily Log, and Profile. Include a fixed bottom navigation suitable for mobile web.
```

Done when

- You can navigate between 4 pages
- Layout works well on mobile width

Step 4 — Build Home page with mock data

Goal

Get a visually strong first screen.

What to build

- Greeting header
- 3 summary cards
- Recent daily log preview
- Quick actions section

Codex prompt

```
Create a mobile-first Home dashboard page in React + TypeScript using Tailwind CSS. Include a greeting header, three summary cards, a recent daily log section, and quick action buttons. Make it look soft, calm, and modern.
```

Done when

- Home page looks polished
- Can be shown on desktop browser in mobile view

Step 5 — Build Goals page with mock data

Goal

Create a usable goal management screen.

What to build

- Goal cards
- Progress bar
- Status badge
- Add goal button

Codex prompt

```
Create a mobile-first Goals page in React + TypeScript using Tailwind CSS. Show goal cards with title, category, progress bar, current value, target value, and status badge. Add a prominent mobile-friendly Add Goal button.
```

Done when

- Goals page displays realistic mock cards
- Layout is clean on phone width

Step 6 — Build Daily Log page with mock form

Goal

Create the daily journal input experience.

What to build

- Date input
- Mood selector
- Energy selector
- Stress selector
- Note textarea
- Save button

Codex prompt

```
Create a mobile-first Daily Log page in React + TypeScript using Tailwind CSS.
Include fields for date, mood, energy, stress, and note. Use local component
state and make the form clean and touch-friendly.
```

Done when

- Daily Log page can accept mock input
- Save button works in local state for now

Step 7 — Build Profile page with mock data

Goal

Complete the main navigation set.

What to build

- Profile summary card
- Name, email, phone
- Small settings shortcuts

Codex prompt

```
Create a mobile-first Profile page in React + TypeScript using Tailwind CSS.
Show a user card, basic contact information, and simple settings shortcut cards.
Keep the style consistent with the rest of the app.
```

Done when

- `4` -page navigation is complete
- The app feels coherent

Step 8 — Clean up types and service structure

Goal

Prepare the app for real backend integration.

What to build

- Shared TypeScript models
- API response types
- Service placeholders

Suggested models

- `User`
- `Goal`
- `DailyLog`
- `ApiResponse<T>`

Codex prompt

```
Create TypeScript models and service placeholders for User, Goal, DailyLog, and
generic API responses. Organize them into types and services folders for a
scalable React app.
```

Done when

- Mock pages use typed models
- Service files exist and compile cleanly

Step 9 — Create Google Sheet database

Goal

Prepare the backend data store.

Manual tasks

Create one Google Sheet with tabs: - users - goals - daily_logs

Add column headers exactly as defined in the Data Model section.

Done when

- All 3 sheets exist
- Headers are correct
- You have one demo user row

Step 10 — Create Google Apps Script backend

Goal

Expose the Sheet data through a simple API.

What to implement

- `doGet(e)`
- `doPost(e)`
- JSON output helper
- Action router

Required actions

GET

- `getUser`
- `listGoals`
- `listDailyLogs`

POST

- `createGoal`
- `createDailyLog`
- `updateProfile`

Codex prompt

```
Generate a Google Apps Script backend for a Google Sheets database. Implement doGet and doPost, return JSON, and support these actions: getUser, listGoals, listDailyLogs, createGoal, createDailyLog, updateProfile. Keep the code organized with helper functions.
```

Done when

- Apps Script deploys as a Web App
- API returns valid JSON in browser/Postman

Step 11 — Connect React to Apps Script API

Goal

Replace mock data with real data.

What to build

- `api.ts` fetch wrapper
- `goals.service.ts`
- `logs.service.ts`
- `profile.service.ts`
- loading and error states

Codex prompt

```
Create a typed fetch-based API service layer for a React + TypeScript app that connects to a Google Apps Script backend. Add methods for getUser, listGoals, listDailyLogs, createGoal, createDailyLog, and updateProfile. Include proper error handling.
```

Done when

- Home, Goals, Daily Log, and Profile load real data
- Create Goal works
- Create Daily Log works

Step 12 — Add forms and submit flows

Goal

Make MVP interactive.

What to build

- Add Goal dialog or sheet
- Daily Log submit flow
- Profile update flow
- Toast or inline status messages

Codex prompt

```
Implement form submission flows for Add Goal, Create Daily Log, and Update
Profile in a mobile-first React app. Add loading state, disabled buttons during
submit, and success/error feedback.
```

Done when

- User can create/update key records
- Feedback is visible after submit

Step 13 — Mobile polish pass

Goal

Make it feel good on real phones.

Checklist

- Increase tap targets
- Check text sizes
- Reduce clutter
- Ensure forms fit small screens
- Make bottom nav stable
- Test on Chrome mobile emulator and real phone

Codex prompt

```
Review this React + Tailwind app for mobile UX. Improve spacing, tap target sizes, form layout, card sizing, and bottom navigation so it feels polished on small screens.
```

Done when

- App is comfortable on phone width
- No obvious overflow or cramped layout

Step 14 — Deploy frontend to Cloudflare Pages

Goal

Publish the web app for free.

Manual tasks

1. Push code to GitHub
2. Create Cloudflare Pages project
3. Connect the GitHub repo
4. Set build command
5. Set output directory
6. Add environment variable for API base URL if needed

Suggested settings

- Build command: `npm run build`
- Output dir: `dist`

Done when

- Public URL works
- Site loads from phone browser

Step 15 — Post-MVP next steps

Do these only after MVP is live.

Next features

- Appointment page

- Better charts
- Real auth
- Role separation
- Admin view
- Search/filter
- Export features
- Better validation

Codex Working Rules

Use these rules when asking Codex to work on the project.

Rule 1

Do one step at a time.

Rule 2

After each step, run the app and confirm it still works.

Rule 3

Do not ask Codex to build the whole project in one prompt.

Rule 4

Always ask for: - complete files - type-safe code - minimal dependencies - mobile-first UI

Rule 5

If Codex changes multiple files, review the diff before accepting.

Recommended Codex Prompt Template

Use this template every time.

```
You are helping me build a mobile-first wellness web app called Happy Balance.
Stack:
- React
```

```
- Vite
- TypeScript
- Tailwind CSS
- Google Apps Script backend
- Google Sheets database

Current step:
[PASTE CURRENT STEP HERE]

Requirements:
- Keep the code type-safe
- Keep the UI mobile-first
- Keep components clean and reusable
- Do not introduce unnecessary dependencies
- Explain which files you create or modify
- Keep the solution scoped only to this step
```

Tonight Checklist

Use this as the execution sequence tonight.

Must finish tonight

- [ ] Create Vite project
- [ ] Set up Tailwind
- [ ] Add routing
- [ ] Build app shell
- [ ] Build Home page
- [ ] Build Goals page
- [ ] Build Daily Log page
- [ ] Build Profile page
- [ ] Create Google Sheet tabs
- [ ] Create Apps Script API
- [ ] Connect at least one page to real data
- [ ] Push GitHub
- [ ] Deploy to Cloudflare Pages

Nice to have tonight

- [ ] Add goal form
- [ ] Add daily log submit
- [ ] Add toast feedback
- [ ] Improve theme

MVP Definition of Done

The MVP is done when: - The app is publicly accessible by URL - It opens well on mobile page loads real user data - Goals page loads from Google Sheets - Daily Log can save a re shows real data - The codebase is structured for the next phase

---

Final Advice

Keep the first version small. Do not try to recreate the entire storyboard at once. Ship the MVP, then grow it.

The fastest winning path is: - beautiful frontend first - simple backend second - real data third deployment